

CSC215 Lab9 – Buffer Overflow

Total: 100pt

@Copyright Dr. Xiang Fu. All Rights Reserved.

What to turn in: CSC215Lab9.docx

Note: the folder used for this lab is located in ~/Desktop/csc215/lectures/lec20_bo_practice in the Ubuntu image.

Before proceeding to the implementation, please do a “sudo make clean” and then “sudo make” FIRST!!!

1. Shell Code Exploration [20 minutes, 30pt]

Use GDB to debug into shell code (shellcode.cc) and find out the values of the parameters for the following system calls:

(1) [2pt] int 80 – service code 0x17. Sys setuid parameter 1: **0x0**

(2) [10pt] int 80 – service code 0xb. execve parameter 1: **0x0804a06e** ,
parameter 2: (**0x0804a06e, 0x0**), parameter 3: (**0x0804a06e, 0x0, 0x0**)(all your parameters should be inform of 32-bit hexadecimal numbers).

Specifically, parameter 2 and parameter 3 are both an array of strings (char *). Use GDB to find out the contents of this two arrays:

parameter 2: {**“/bin/sh”**, **NULL**} (argument list)

parameter 3: **null** (environment list)

(3) [18pt] Explain the purpose of the “zigzag” code of the JMP and CALL sequence in shellcode.

The JMP/CALL “zigzag” technique is used in shellcode to obtain the *runtime memory address* of embedded data (usually the “/bin/sh” string) without using any hard-coded addresses.

2. [20 minutes, 30pt] Replicate the 3-section Perl command attack on meet.cc. Paste the exploit

screen below.

[illegible]

3. [30 minutes, 30pt] Use [exploit.cc](#) to generate a working payload on meet.cc. Paste the exploit screen to show your success. [hint: you need GDB to find out the proper size and offset.].

```

20(220(220(220(220(220 ...)) at meet.cc:6
6      strcpy(name, temp2);
(gdb) x/4wx $ebp
0xbffff278:      0xbffff298      0x08048563      0xbffff4b5      0xbffff4b8
(gdb) n
7      printf("hello %s %s\n", temp1, temp2);
(gdb) x/4wx $ebp
0xbffff278:      0xbffff4b9      0xbffff4b9      0xbfff00b9      0xbffff4b8
(gdb) c

```

4. [10 minutes, 10pt] Describe a protection scheme that could be used by firewalls to defend the raw buffer overflow attacks like the ones in questions 2 and 3 (i.e., without the use of packers applied on payload).

A firewall can implement strict validation of protocol specifications and field sizes to block raw buffer-overflow attempts. By enforcing maximum allowed lengths for elements such as HTTP headers, CGI parameters, and other application-level fields, the firewall can automatically discard any input that exceeds these limits. This prevents oversized or malformed data from ever reaching the vulnerable application and helps stop buffer-overflow attacks before they occur.